

Sistema de Gestión de Rutas y Paquetes Empresa de Mensajería

Reporte Técnico

Asignatura: Estructuras de Datos
Programación Orientada a Objetos en C++

1. Introducción

El presente reporte documenta el diseño, implementación y funcionamiento del Sistema de Gestión de Rutas y Paquetes para una empresa de mensajería. El sistema fue desarrollado en C++17 bajo el paradigma de Programación Orientada a Objetos, con una arquitectura modular compilada con CMake.

El objetivo principal es automatizar los procesos internos de la empresa: registro de ciudades en rutas, administración de paquetes según su urgencia, simulación del desplazamiento de un camión y almacenamiento de distancias y tiempos mediante distintas estructuras matriciales.

Objetivos específicos:

- Implementar una lista enlazada para modelar las ciudades de una ruta.
- Usar una cola (FIFO) para gestionar paquetes pendientes de asignación.
- Usar una pila (LIFO) para gestionar paquetes urgentes con prioridad de salida.
- Simular el recorrido continuo de un camión mediante una cola circular.
- Almacenar distancias (vector 1D) y tiempos (matriz 2D) entre ciudades.
- Registrar rutas prioritarias con una matriz dispersa.
- Organizar el proyecto de forma modular compilable con CMake.

2. Descripción del Problema

La empresa requiere centralizar la gestión de sus operaciones logísticas. Los procesos manuales actuales no escalan con el crecimiento del volumen de paquetes y rutas. El sistema resuelve las siguientes necesidades:

Necesidad	Estructura	Justificación
Registro dinámico de ciudades	Lista Enlazada	Inserción/eliminación sin redimensionar
Paquetes pendientes	Cola (FIFO)	El primero en llegar, primero en salir
Paquetes urgentes	Pila (LIFO)	El más reciente urgente sale primero
Recorrido del camión	Cola Circular	Avance cíclico entre puntos de control
Distancias entre puntos	Arreglo 1D	Acceso directo por índice
Tiempos entre ciudades	Matriz 2D (NxN)	Consulta $O(1)$ por par origen-destino
Rutas de alta prioridad	Matriz Dispersa	Solo almacena entradas no cero

3. Diseño de Clases

El sistema está compuesto por 6 clases organizadas en tres capas: clases entidad (Ciudad, Paquete), clases gestoras (Ruta, GestorPaquetes, GestorRutas) y la clase integradora (SistemaMensajería).

3.1 Descripción de clases

Clase	Responsabilidad	Estructuras internas
Ciudad	Nodo de la lista enlazada de ruta	Atributos: nombre, id, *siguiente
Ruta	Lista enlazada de ciudades	Puntero cabeza, contador de nodos
Paquete	Datos de un paquete	id, descripcion, destino, urgente
GestorPaquetes	Administra cola y pila de paquetes	std::queue y std::stack de Paquete
GestorRutas	Matrices y cola circular	vector 1D/2D, vector disperso, array circular
SistemaMensajería	Integra todo el sistema	Composición de Ruta, GestorPaquetes, GestorRutas

3.2 Relaciones entre clases (UML textual)

```
SistemaMensajería
+-- Ruta (composicion)
| +-- Ciudad* cabeza --> Ciudad --> Ciudad --> ...
+-- GestorPaquetes (composicion)
| +-- queue[Paquete] (cola pendientes FIFO)
| +-- stack[Paquete] (pila urgentes LIFO)
+-- GestorRutas (composicion)
+-- vector[double] (distancias 1D)
+-- vector[vector[double]] (tiempos 2D)
+-- vector[EntradaDispersa] (matriz dispersa)
+-- vector[string] circular (cola circular camion)
```

4. Estructuras de Datos Utilizadas

4.1 Lista Enlazada — Clase Ruta

Cada nodo es un objeto Ciudad con un puntero al siguiente. Permite agregar y eliminar ciudades en tiempo $O(n)$ sin reasignar memoria contigua.

```
// Agregar al final
Ciudad* nueva = new Ciudad(nombre, contadorId++);
Ciudad* actual = cabeza;
while (actual->siguiente != nullptr) actual = actual->siguiente;
actual->siguiente = nueva;
```

4.2 Cola (FIFO) — Paquetes Pendientes

Se utiliza `std::queue` de la STL. El primero en registrarse es el primero en procesarse, respetando el orden de llegada.

```
colaPendientes.push(p); // encolar
Paquete p = colaPendientes.front();
colaPendientes.pop(); // desencolar
```

4.3 Pila (LIFO) — Paquetes Urgentes

Se utiliza `std::stack` de la STL. El último urgente registrado es el primero en procesarse, modelando atención de emergencia inmediata.

```
pilaUrgentes.push(p); // apilar
Paquete p = pilaUrgentes.top();
pilaUrgentes.pop(); // desapilar
```

4.4 Cola Circular — Simulación del Camión

Implementada sobre un vector con índices 'frente' y 'fin' que avanzan en módulo de la capacidad. Al avanzar, el punto actual se reinserta al fondo, simulando un recorrido cíclico continuo.

```
void avanzarCamion() {
    string actual = puntosControl[frente];
    frente = (frente + 1) % capacidad;
    fin = (fin + 1) % capacidad;
    puntosControl[fin] = actual; // reinsertar
}
```

4.5 Matrices: 1D, 2D y Dispersa

El sistema emplea tres formas de almacenamiento matricial con propósitos distintos:

- Matriz 1D (vector): distancias entre puntos relevantes. Acceso $O(1)$ por índice.
- Matriz 2D (vector de vectores $N \times N$): tiempos entre ciudades. Acceso $O(1)$ por (i,j) .
- Matriz dispersa (vector de tripletas fila/columna/valor): rutas prioritarias. Solo almacena entradas no cero, eficiente cuando la mayoría de pares no tienen ruta asignada.

```
struct EntradaDispersa {
    int fila, columna;
    double valor;
};
```

```
vector<EntradaDispersa> matrizDispersa;
```

5. Estructura del Proyecto y CMake

El proyecto sigue una organización modular estándar con separación de interfaces (/include) e implementaciones (/src). Cada clase ocupa un par de archivos .h y .cpp independientes.

```
/ProyectoMensajeria
CMakeLists.txt
/include
Ciudad.h Ruta.h Paquete.h
GestorPaquetes.h GestorRutas.h SistemaMensajeria.h
/src
Ciudad.cpp Ruta.cpp Paquete.cpp
GestorPaquetes.cpp GestorRutas.cpp
SistemaMensajeria.cpp main.cpp
```

Contenido del CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.15)
project(ProyectoMensajeria VERSION 1.0 LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
include_directories(${PROJECT_SOURCE_DIR}/include)
set(SOURCES
src/main.cpp src/Ciudad.cpp src/Ruta.cpp
src/Paquete.cpp src/GestorPaquetes.cpp
src/GestorRutas.cpp src/SistemaMensajeria.cpp)
add_executable(mensajeria ${SOURCES})
```

Comandos para compilar y ejecutar:

```
mkdir build && cd build
cmake ..
make
./mensajeria
```

6. Resultados del Programa en Ejecución

A continuación se muestran las salidas reales del sistema al ejecutar cada módulo con los datos de demostración precargados.

6.1 Inicio y carga de datos

```
=====
SISTEMA DE GESTION DE MENSAJERIA v1.0
=====
Cargar datos de demostracion? (s/n): s
[+] Ciudad 'Ciudad Victoria' agregada a la ruta.
[+] Ciudad 'Monterrey' agregada a la ruta.
[+] Ciudad 'Saltillo' agregada a la ruta.
[+] Ciudad 'San Luis Potosi' agregada a la ruta.
[+] Paquete #1 agregado a la cola de pendientes.
[+] Paquete #2 agregado a la cola de pendientes.
[+] Paquete urgente #3 apilado.
[+] Paquete urgente #4 apilado.
[+] Matriz 2D inicializada para 4 ciudades.
[+] Cola circular inicializada con capacidad 6.
```

6.2 Resumen general del sistema

```
[RUTA ACTUAL]
1.[ID:1] Ciudad Victoria --> 2.[ID:2] Monterrey
--> 3.[ID:3] Saltillo --> 4.[ID:4] San Luis Potosi

[ESTADO DE PAQUETES]
Cola pendientes: 2 paquete(s)
Pila urgentes: 2 paquete(s)
Paquete #1 | Destino: Monterrey | Electrodomesticos [normal]
Paquete #4 | Destino: Monterrey | Material quirurgico [URGENTE]

[POSICION DEL CAMION]
Posicion actual: 'Base Central'
```

6.3 Matriz 2D de tiempos y matriz dispersa

```
-- Matriz 2D: Tiempos estimados (horas) --
Victoria Monterrey Saltillo SLP
Victoria 0 2.5 - 6
Monterrey 2.5 0 1 -
Saltillo - 1 0 3.5
SLP 6 - 3.5 0

-- Matriz Dispersa: Rutas Prioritarias --
Origen Destino Prioridad
Victoria Monterrey 9.5
Monterrey SLP 8.0
Victoria Saltillo 7.2
```

6.4 Compilación con CMake

```
[ 12%] Building CXX object ... src/main.cpp.o
[ 25%] Building CXX object ... src/Ciudad.cpp.o
[ 37%] Building CXX object ... src/Ruta.cpp.o
[ 50%] Building CXX object ... src/Paquete.cpp.o
[ 62%] Building CXX object ... src/GestorPaquetes.cpp.o
[ 75%] Building CXX object ... src/GestorRutas.cpp.o
```

```
[ 87%] Building CXX object ... src/SistemaMensajeria.cpp.o
[100%] Linking CXX executable mensajeria
[100%] Built target mensajeria
```


7. Conclusiones

- La Programación Orientada a Objetos permitió dividir el sistema en clases con responsabilidad única, facilitando la mantenibilidad y la extensión del código.
- La elección de cada estructura de datos responde a la semántica del problema: la cola refleja el orden de llegada, la pila refleja la urgencia inmediata y la cola circular modela el recorrido continuo del camión.
- La matriz dispersa es más eficiente que una matriz $N \times N$ completa cuando la mayoría de los pares origen-destino no tienen rutas prioritarias asignadas, ya que solo almacena las entradas con valor no cero.
- La organización modular con CMake facilita la compilación incremental, la detección temprana de errores y la integración con herramientas de desarrollo.
- La lista enlazada implementada manualmente permite agregar y eliminar ciudades sin redimensionar memoria, lo cual es adecuado para rutas que cambian con frecuencia.